SOFTWARE ENGINEERING / PROJECT REPORT

Cinema Booking System

End-to-End Development Report

A structured record of Phases 1–9: APIs, database, authentication, UI, testing and cloud deployment.

PROJECT TYPE Educational full-stack web application

TECHNOLOGY Vue 3 · Node.js · Express · SQLite

REPOSITORY github.com/CHS6868/cinema-booking-system

PREPARED 5 November 2026

DEVELOPMENT RECORD Nine phases | October–November 2026

Scope statement

This is a demonstration booking system, not a commercial ticketing platform. It does not process real payments, issue valid tickets or represent a live cinema inventory.

Document guide

This report explains what was built, why each phase mattered, how the parts interact, and what the developer validated. The diagrams are explanatory schematics; they are not claims of complete code coverage, production certification or formal UML conformance.

Contents

01 Executive summary and scope

02 Requirements and project decisions

03 Solution architecture and data model

04 Phase-by-phase implementation (1–9)

05 API design and booking business rules

06 Verification, quality and security

07 Deployment and repository evidence

08 Challenges, lessons and future work

09 Appendix: files, commands and sources

Evidence and interpretation

The material was reconstructed from the project source snapshots supplied during development, the final README, the Phase 8 test plan, project discussions, and screenshots of the GitHub repository and Railway dashboard. A screenshot showing “Completed” on Railway records deployment completion, but is not a substitute for independent security or public end-to-end verification.

What the report does not claim The document does not invent a public Railway URL, production uptime, real customers, money collected, exact coverage percentages or a published v1.0.0 GitHub release.

01 Executive summary and scope

The Cinema Booking System is a full-stack educational web application that allows a registered user to browse films, choose showtimes, view available seats, create demo reservations, review personal bookings and cancel eligible reservations. Its primary purpose is to build hands-on software engineering skills across the complete development lifecycle.

The application evolved from a small Express API and Vue user interface into a database-backed system with authentication, transaction-safe booking logic, modular frontend components, automated tests and a Railway deployment configuration. The project was implemented in VS Code using a lightweight JavaScript toolchain suited to a modest laptop.

Project objectives

- Implement an intuitive booking flow from movie selection to reservation confirmation.
- Persist cinema data and reservations in a relational database while preventing duplicate confirmed seats.
- Associate bookings with authenticated users and enforce ownership boundaries.
- Support cancellation without deleting booking history, and release seats consistently.
- Improve maintainability through reusable Vue components, automated tests and documentation.
- Demonstrate production build and deployment concepts using a cloud-hosted single-service architecture.

Scope boundaries

Included in the demo	Excluded or deferred
Movie catalogue, showtime and hall data	Real cinema inventory feeds

Included in the demo	Excluded or deferred
Seat reservation and cancellation	Real payment processing and refunds
Registration, login and sessions	Email verification and password recovery
Single-instance SQLite persistence	High-scale multi-instance production design
Automated and manual testing	External penetration-test certification

02 Requirements and project decisions

Functional and non-functional requirements

Area	Requirement / design response
Browsing	Show movies, metadata and only relevant upcoming showtimes.
Seat selection	Provide seat availability and a clear maximum-selection rule.
Booking integrity	Reject invalid seat choices, expired showtimes and duplicate confirmed seats.
Access control	Protect booking and history endpoints with authenticated sessions and ownership checks.
Cancellation	Allow cancellation only for the owner of a future confirmed reservation.
Usability	Keep the same core workflow usable on desktop, tablet and mobile viewport widths.
Quality	Enable repeatable unit, API and frontend component tests.
Deployment	Serve SPA and APIs from one origin, with persistent database storage.

Technology selection rationale

Vue 3 and Vite enable fast frontend development and componentization. Express offers a straightforward REST API layer. SQLite is suitable for a small demonstration workload and allows the developer to practice SQL constraints and transactions without maintaining a separate database service. Node.js supplies a common JavaScript environment across application, scripts and tests.

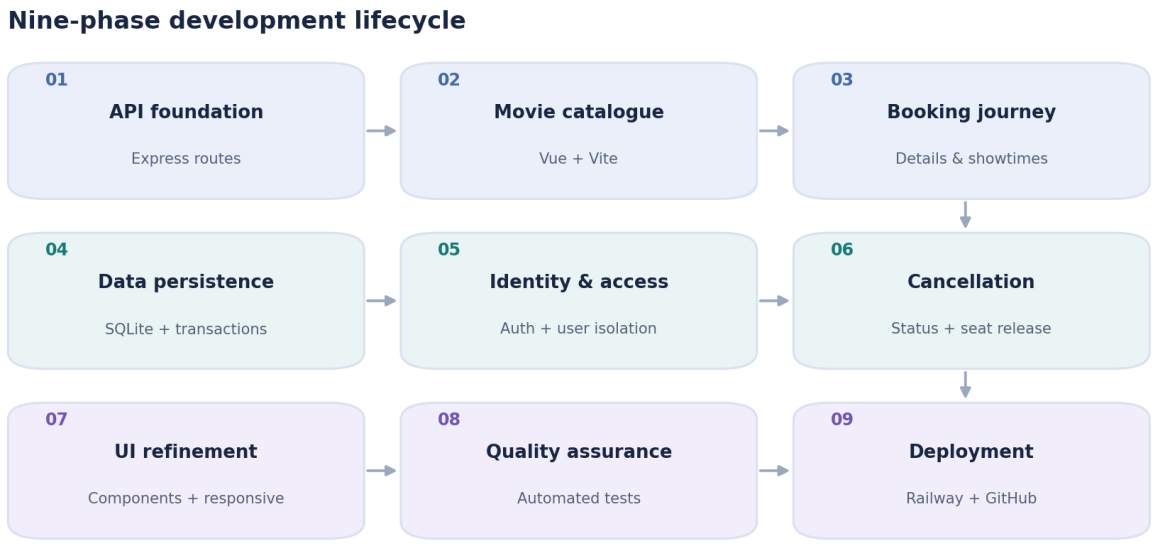


Figure 1. Nine-phase progression (sequence of milestones; not a duration or effort chart).

03 System architecture and data model

The application uses a client/server design. In local development, Vite serves the frontend and proxies /api requests to Express. In production, Express serves compiled Vue assets and the REST endpoints from the same origin. SQLite stores user accounts, sessions, movies, halls, showtimes, bookings and seat allocations.

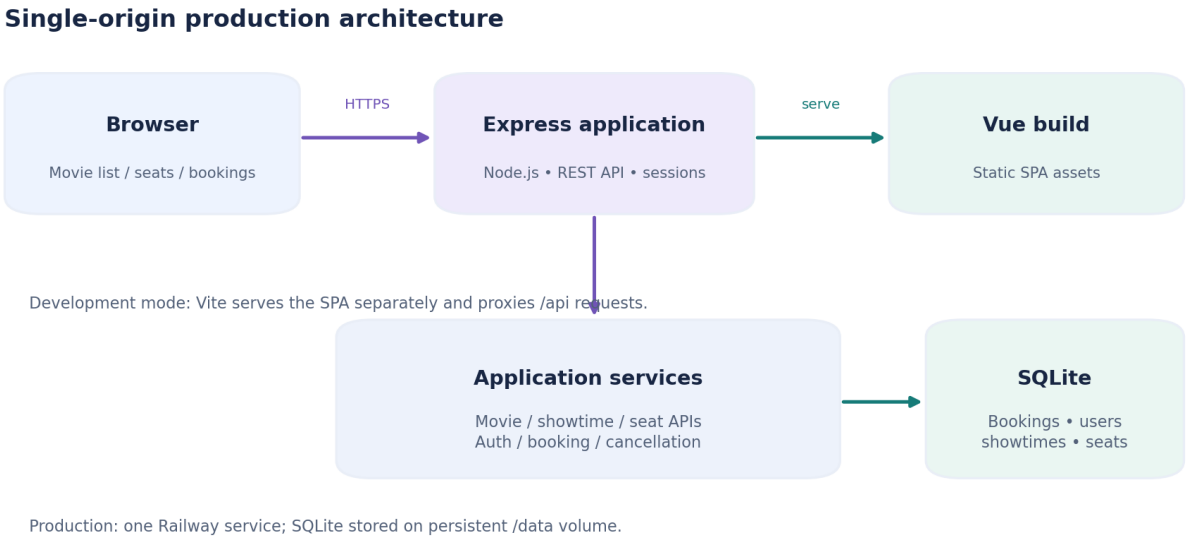


Figure 2. Logical architecture of the final full-stack demonstration application.

Relational data model

The schema was introduced in Phase 4 and extended for account sessions and cancellation. Important relational relationships are: one movie has many showtimes; one hall has many seats and showtimes; one registered user can have many bookings; a booking references a showtime and one or more seat allocation rows.

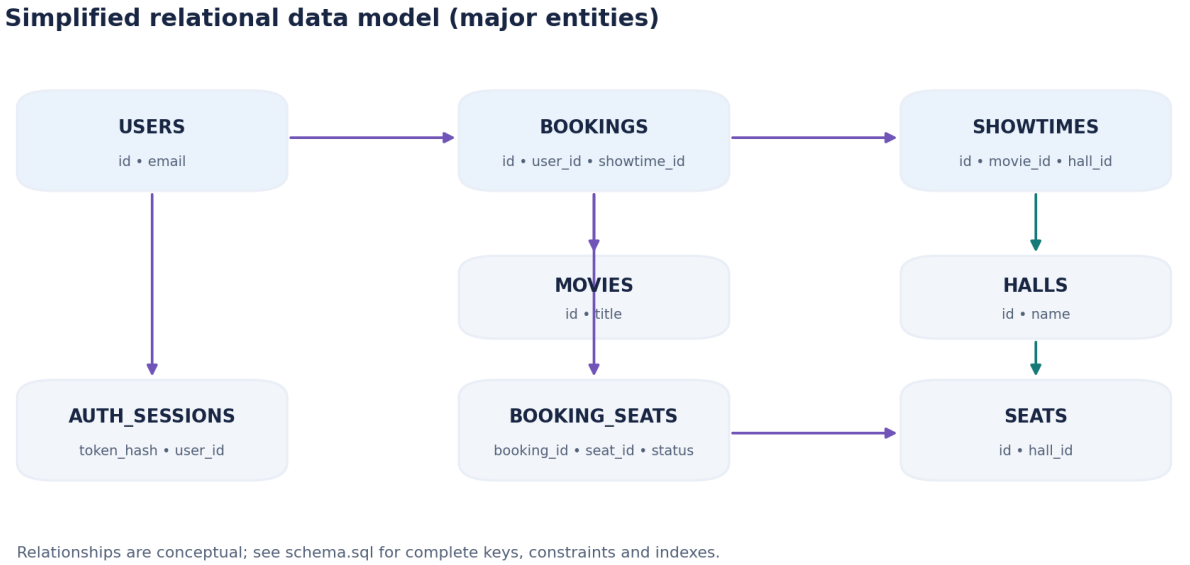


Figure 3. Simplified relational entities and dependencies; the actual schema remains authoritative.

Database integrity mechanisms

- SQLite foreign keys connect reservations to valid parent entities.
- Bookings and booked seats are inserted inside a BEGIN IMMEDIATE / COMMIT transaction, with ROLLBACK on failure.
- A partial unique index on (showtime_id, seat_id) for CONFIRMED seat rows helps prevent double reservations.
- Cancellation updates both booking and seat statuses in one transaction; the historical records remain available.

04 Phase-by-phase implementation

The nine phases were incremental: each one introduced a working slice of the system and provided a foundation for the next stage. The following sections describe the achieved functionality, its software engineering purpose and the principal verification approach.

Phase 1 — Backend API foundation

Goal. Build the first working service and define the API boundary.

- Established a Node.js/Express application with development-time startup in VS Code and a local HTTP port.
- Introduced JSON REST endpoints and initial demonstration movie data so requests could be tested independently of Vue.
- Used the backend as the authoritative place for movie and booking rules, rather than putting all logic in the browser.

Outcome / evidence. The initial service could respond to basic requests, forming the base for subsequent API and database integration.

Engineering learning. API routing, HTTP methods/status codes and the separation of frontend from backend.

Phase 2 — Vue application and movie catalogue

Goal. Create a navigable user interface for browsing films.

- Scaffolded the Vue 3 + Vite frontend and added Vue Router for page navigation.
- Built the homepage/movie listing and connected it to Express via the local Vite /api proxy.
- Added movie cards and links so a user could move from browsing to movie details.

Outcome / evidence. Movie cards rendered successfully and navigation began to work end-to-end.

Engineering learning. Frontend routing, asynchronous fetch requests and client/server integration.

Phase 3 — Movie details and showtimes

Goal. Convert the catalogue into a usable selection flow.

- Introduced a dedicated movie-details page with movie metadata and available showtime options.
- Added date/time selection and navigation toward the seat-selection screen.
- Fixed navigation issues and aligned the Vue views with backend showtime identifiers.

Outcome / evidence. Users could select a movie and an available date/time before proceeding to choose seats.

Engineering learning. Reusable page states, route parameters and consistent identifiers across API boundaries.

Phase 4 — SQLite database integration

Goal. Move from ephemeral data to persistent relational records.

- Created and seeded SQLite tables for movies, halls, seats, showtimes, bookings, booking_seats and users.
- Replaced temporary in-memory data paths with parameterized SQLite queries for movie and seat APIs.
- Used BEGIN IMMEDIATE transactions and a confirmed-seat unique index to prevent double bookings.
- Added seed and database inspection scripts (3 movies, 3 halls, 72 seats).

Outcome / evidence. New bookings survived backend restart; same-showtime seat conflicts were rejected and foreign-key checks were used.

Engineering learning. Schema design, referential integrity, concurrency-sensitive transactions and migration troubleshooting.

Phase 5 — User authentication and booking ownership

Goal. Ensure bookings belong to identifiable accounts.

- Implemented registration and login with scrypt password hashing and verification.
- Created SQLite-backed sessions using random tokens; only token hashes are stored in the database.
- Protected personal booking endpoints with requireAuth and used req.user.id to associate bookings with their owner.
- Built register/login screens, authentication-aware navigation and My Bookings; tested account separation in independent browser sessions.

Outcome / evidence. Account A and Account B were confirmed to see only their own records; a shared-cookie browser-tab confusion was resolved during testing.

Engineering learning. Authentication versus authorization, cookie/session handling and user-specific data access.

Phase 6 — Booking cancellation and seat release

Goal. Support a complete reservation lifecycle without deleting history.

- Introduced PATCH /api/bookings/:id/cancel behind the authentication middleware.
- Enforced booking ownership, CONFIRMED status and a future-showtime rule before allowing cancellation.
- Updated booking and booking_seats rows together in a transaction, then re-exposed released seats for booking.
- Debugged a repeated Cannot PATCH 404, ultimately verifying that the router worked when the correct backend process was running.

Outcome / evidence. The cancellation route, state transition, repeated-cancel rejection and released-seat rebooking were exercised in the test workflow.

Engineering learning. Business-rule authorization, transactional state changes and debugging stale local server processes.

Phase 7 — Component refactoring and responsive UI

Goal. Improve maintainability, presentation and small-screen usability.

- Reorganized the frontend around reusable MovieCard, ShowtimeCard, SeatMap, BookingSummary and Navbar components.
- Improved navigation, loading/error handling, booking cards and responsive seat selection.
- Added local movie poster assets and client-side Upcoming / Past / Cancelled booking filters.
- Used Chrome DevTools responsive mode to check mobile, tablet and desktop layouts without requiring physical devices.

Outcome / evidence. The UI was rebuilt and tested on representative viewport widths; remaining changes were handled as targeted refinements.

Engineering learning. Component contracts, UI consistency, responsive CSS and regression awareness.

Phase 8 — Software testing and QA

Goal. Make quality checks repeatable rather than entirely manual.

- Added backend unit tests with node:test, plus API integration tests for registration, login, reservation, cancellation and authorization.
- Added Vitest and Vue Test Utils cases for SeatMap and BookingSummary component behavior.
- Introduced coverage commands, a test plan and manual regression checks for desktop/mobile layouts and user isolation.
- Resolved test-compatibility defects involving a missing showtime helper, duplicate session fixture creation and Express app export.

Outcome / evidence. A prior development snapshot recorded 26 passing backend checks and 0 failures; current production-grade coverage percentages were not provided and are not invented here.

Engineering learning. Automated regression design, isolated test databases, fixtures, testability and the difference between coverage and correctness.

Phase 9 — Production build, Railway and GitHub portfolio

Goal. Demonstrate a deployment-ready, documented full-stack project.

- Configured a single-service production architecture with Express serving built Vue assets and API routes.
- Added a production startup script, environment-based port/database configuration and Railway build/start commands.
- Planned persistent SQLite storage on a Railway volume, instead of relying on a disposable application filesystem.
- Updated the GitHub README, About description and topic tags; the supplied GitHub screenshot shows the Phase 9 completion commit.

Outcome / evidence. Railway displayed a Completed deployment status in a supplied screenshot; public HTTPS functionality and the current domain state are not independently certified by this report.

Engineering learning. Build versus runtime, hosting configuration, persistence, documentation and responsible public-demo disclosure.

05 API design and booking business rules

The API applies the principle that business rules must be checked by the server, even if the frontend disables invalid actions. The table below summarizes major endpoints across Phases 1–6 and the subsequent testing stages.

Method	Endpoint	Purpose / protection
GET	/api/movies	Movie catalogue and upcoming counts
GET	/api/movies/:id	Film details
GET	/api/movies/:id/showtimes	Available dates and times
GET	/api/showtimes/:id/seats	Seat map and CONFIRMED availability
POST	/api/auth/register	Validate and create user account
POST	/api/auth/login	Verify password; create session
GET	/api/auth/me	Return session user; protected
POST	/api/auth/logout	Invalidate session
POST	/api/bookings	Create booking; protected; transactional
GET	/api/my-bookings	List only req.user.id bookings
PATCH	/api/bookings/:id/cancel	Owner-only cancellation; transactional

Booking workflow

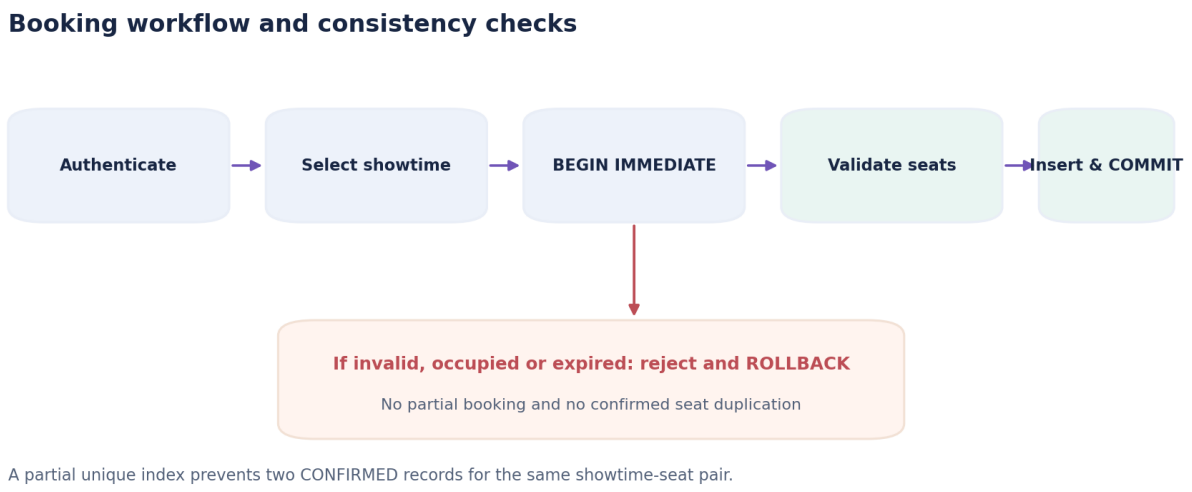


Figure 4. Reservation creation with validation, conflict detection and atomic persistence.

Cancellation workflow

Cancellation state transition

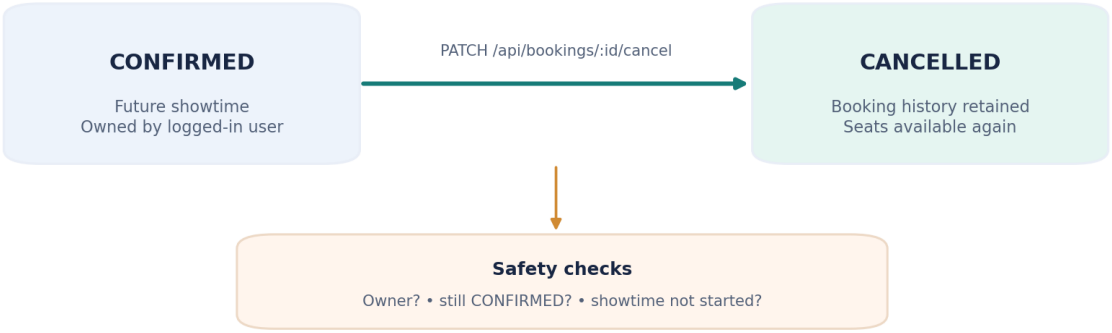


Figure 5. Controlled change from CONFIRMED to CANCELLED and seat release.

Example error and state handling

Condition	Typical server response / effect
Not logged in	401 Unauthorized
Invalid booking/seat input	400 Bad Request
Booking not owned by current user	404 Not Found (avoid exposing another account's record)
Seat already confirmed	409 Conflict
Repeated cancellation	409 Conflict
Showtime already started	400 Bad Request
Transaction failure	ROLLBACK to avoid partial state

06 Verification, quality and security

Test layers

Phase 8 testing strategy - complementary levels

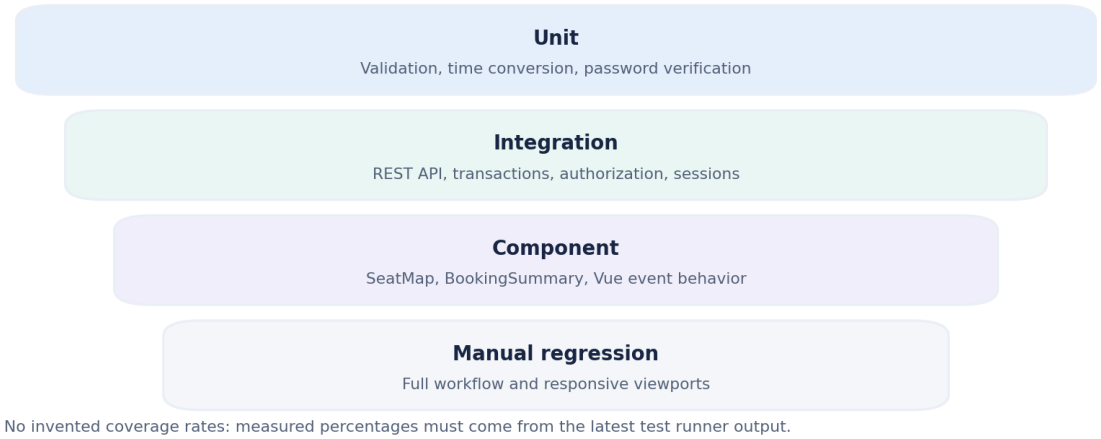


Figure 6. How unit, integration, component and manual testing complement one another.

Representative QA test cases

Case	Test objective	Expected behavior

Case	Test objective	Expected behavior
UT-01	Malaysian showtime conversion	Correct timestamp / invalid inputs rejected
API-02	Registration and duplicate email	Created, then conflict
API-03	Correct and incorrect login	Successful session vs 401
API-06	Double booking	Conflicting confirmed seat rejected
API-07	User isolation	Other user's history hidden, cancellation denied
API-08	Cancel own future booking	State changed; seat becomes available
API-09	Cancel twice	Second attempt rejected
API-10	Rebook released seat	New reservation permitted; FK integrity retained
UI-02	Booked seat in SeatMap	Button disabled
M-01–03	Responsive UI	Usable at 375, 768, 1280 px widths

Test evidence caveat

The Phase 8 kit reported a verified backend snapshot of 26 passing checks and no failures. The QA plan also reserved fields for the developer's final local test counts, frontend results and code-coverage percentages. Those numeric fields were not supplied in the final source material; therefore, this report does not present unsupported accuracy or coverage statistics.

Controls and residual risks

Control already demonstrated	Remaining production consideration
Scrypt password hashes	Password reset and account recovery not implemented
HttpOnly, SameSite cookies; Secure in production	Evaluate CSRF defenses for state-changing requests
Owner-scoped booking queries	Continue testing authorization on every new route
SQLite transactions, partial unique index	Backups, write contention and scale limits
Demo data, no real payments	Rate limiting, logging, abuse monitoring on public domain

07 Deployment and repository evidence

Deployment configuration

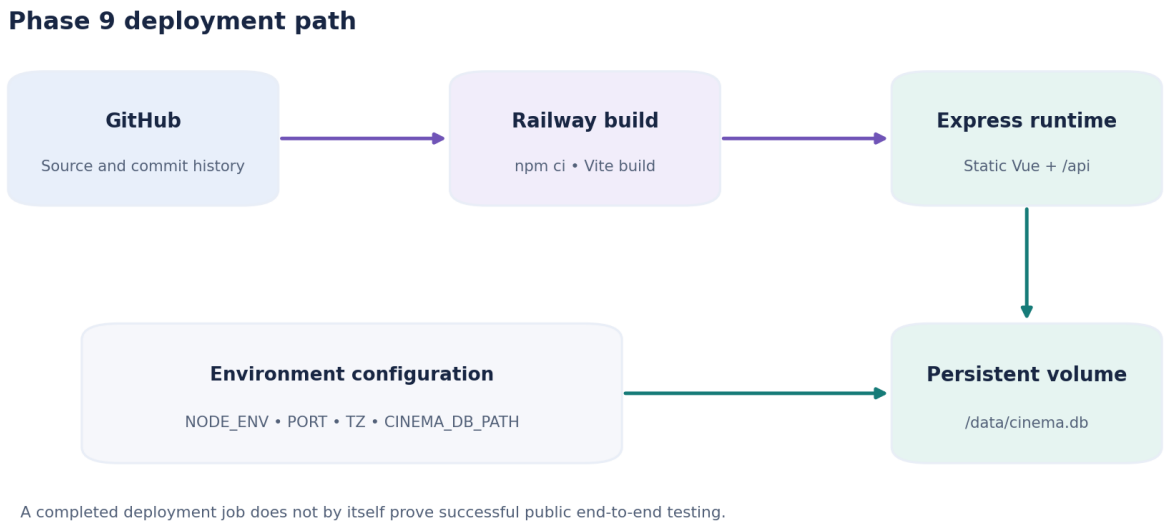


Figure 7. GitHub-to-Railway build and runtime model with persistent SQLite storage.

The final README documents a Railway build command that installs frontend dependencies, builds the Vite application and installs production backend dependencies. The application is started by npm start, invoking a

production startup script. The frontend and API share a single origin, and the SQLite path is configured for a persistent /data volume.

```
Build: npm ci --prefix frontend --include=dev && npm run build --prefix frontend && npm ci --prefix backend --omit=dev
Start: npm start
Data:  CINEMA_DB_PATH=/data/cinema.db
```

Deployment evidence

A user-supplied Railway screenshot dated 4 November 2026 shows a cinema-booking-system service with a “Completed” deployment state. It confirms that the hosting workflow reached its completed state, not that every production user journey was independently tested.

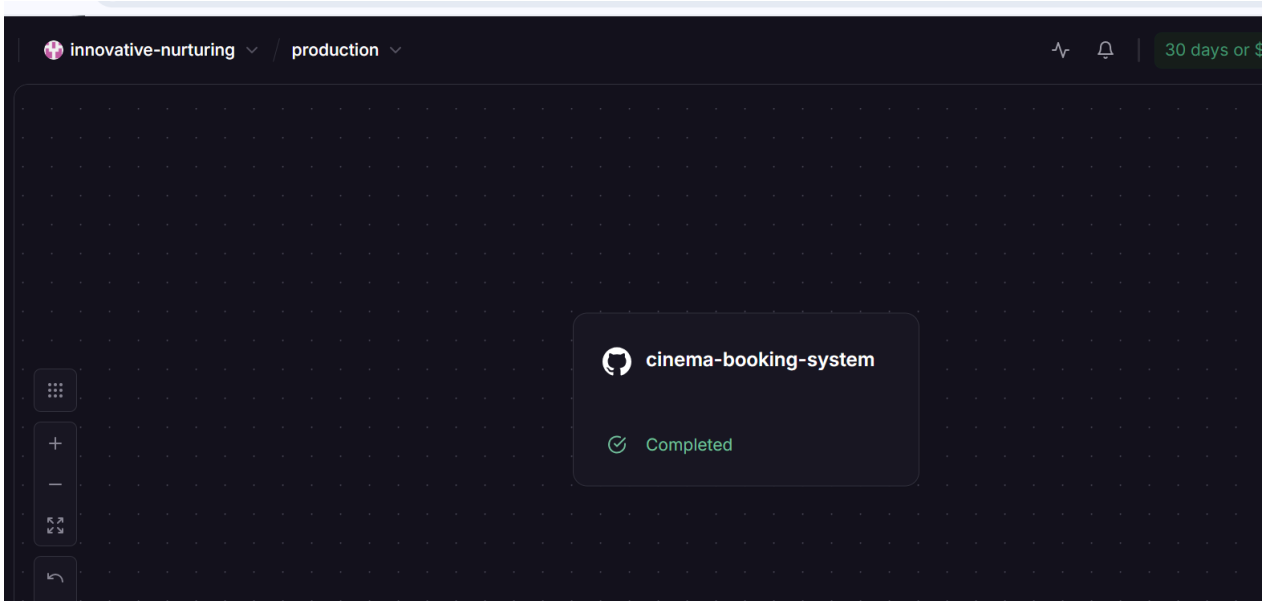


Figure 8. Railway deployment status captured during Phase 9 (user-supplied screenshot).

GitHub evidence

A user-supplied GitHub screenshot dated 5 November 2026 shows the final README.md update and the commit “Complete Phase 9 deployment and portfolio documentation.” It also displays a project description, topic tags and 14 commits at that point in the project history.

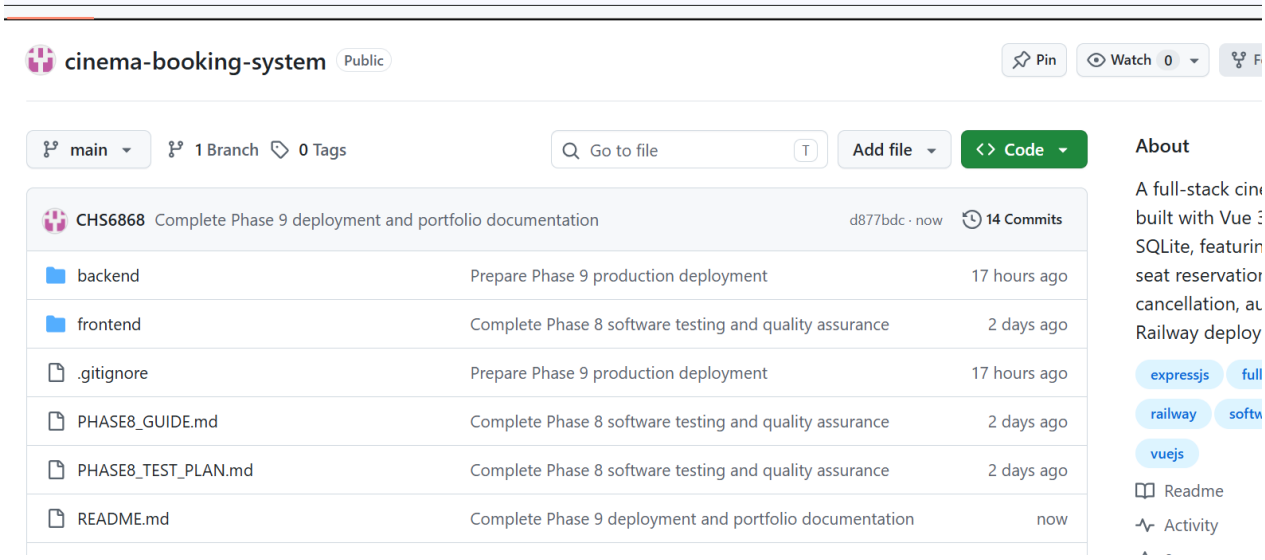


Figure 9. GitHub repository and final Phase 9 completion commit (user-supplied screenshot).

08 Challenges, decisions and lessons learned

Problems encountered and how they were addressed

Challenge	Resolution / software engineering lesson
In-memory versus SQLite showtime IDs	Unified API reads with persisted showtime IDs, reducing frontend/backend inconsistency.
Unsupported db.transaction call	Switched to explicit BEGIN IMMEDIATE / COMMIT / ROLLBACK with node:sqlite.
Repeated Cannot GET/PATCH errors	Diagnosed stale Node processes, route registration and port mismatch instead of assuming a database bug.
Two accounts seemed to show the same bookings	Used separate browser sessions to distinguish shared-cookie state from an authorization defect.
Missing Vue views and import errors	Checked component paths and ensured each router import referred to an existing file.
Phase 8 testing import/fixture failures	Exported Express app for isolated tests and corrected test database setup and missing helper imports.
Showtimes expired after the initial seed	Reran the showtime seeding script and identified rolling scheduling as a future enhancement.
Portfolio commit named Initial commit	Amended final Phase 6 commit and established clearer phase-based Git history.

Software engineering competencies developed

- API design and HTTP-level debugging: validating endpoints, status codes and authentication boundaries.
- Relational data modelling: entities, foreign keys, indexes and consistent transaction behavior.
- Full-stack integration: Vue Router, async requests, cookie/session state and backend responses.
- Refactoring and UI engineering: reusable components, CSS breakpoints and progressive interface improvements.
- Quality assurance: fixtures, repeatable tests, regression checklists and evidence-based reporting.
- Deployment and DevOps fundamentals: builds, environment variables, runtime processes, Git commits and persistent storage.

Future extensions (not part of the finished nine-phase scope)

- Demo checkout/payment simulation, but not real money transfer.
- Admin dashboard for movies, showtimes and halls.
- Automated rolling-showtime generation and archival/maintenance jobs.
- Booking receipts, emails or downloadable demonstration e-tickets.
- Dedicated CSRF defenses, rate limiting, backups, monitoring and accessibility audits.
- A larger database and deployment strategy if usage or concurrent writes outgrow SQLite.

09 Appendix: project artifacts and reproducibility

Repository layout (representative)

```
cinema-booking-system/
  backend/
    auth/              # password and session helpers
    database/          # schema, db.js and seed/init scripts
    routes/            # bookingCancellation.js
    scripts/           # seedShowtimes.js, startProduction.js
    tests/             # unit and API integration suites
    server.js          # REST application
  frontend/
    public/posters/    # demonstration poster assets
    src/components/    # MovieCard, SeatMap, BookingSummary ...
    src/views/         # movie list, details, auth, bookings
    vitest.config.js   # Vue test setup
  PHASE8_TEST_PLAN.md
```

```
README.md          # portfolio documentation
```

Commands used during development

Task	Representative command
Start backend	cd backend; node server.js
Start frontend	cd frontend; npm.cmd run dev
Test backend	cd backend; npm.cmd test
Test frontend	cd frontend; npm.cmd test
Build Vue	cd frontend; npm.cmd run build
Seed upcoming showtimes	cd backend; node scripts/seedShowtimes.js
Review changes	git status / git diff --cached --stat
Publish milestone	git commit -m "..."; git push origin main

Development artifacts consulted

Primary evidence: user-provided source snapshots (Cinema Booking System ZIPs), the final project README, the Phase 8 test plan and installation guide, and user-provided GitHub/Railway screenshots. The GitHub repository is the authoritative place to inspect the latest committed source; earlier snapshots are not assumed to exactly match the final release.

Repository: https://github.com/CHS6868/cinema-booking-system

Concluding assessment

Across nine phases, the project grew from a basic API into a complete educational full-stack demonstration with stateful bookings, controlled cancellation, authentication, tests and deployment documentation. Its strongest engineering outcomes are the progressive separation of concerns, transaction-protected booking operations, user-scoped authorization and explicit quality and deployment workflows. The work is suitable for a software engineering learning portfolio provided it is presented accurately as a demonstration system, not as a commercial ticketing service or an audited production platform.